

第 38 讲

Adam Optimizer

为什么优化器不仅要记住 Gradient 的方向，还要记住 Gradient 的大小

《面向 LLM / Agent Systems 科研的深度学习零基础课》

本讲定位

第 37 讲已经理解 Momentum：用历史 Gradient 的指数加权结果来减少震荡、保留稳定方向。但 Momentum 主要回答“方向历史”问题，还没有解决另一个现实困难：不同参数坐标的 Gradient 尺度可能差很多。Adam（Adaptive Moment Estimation）把 Momentum 的一阶历史与 Gradient 平方的二阶历史结合起来，为不同坐标自动调整有效步长。它也是现代 LLM 训练中理解 AdamW 的直接前置课。

学习目标

学完本讲，你应该能够：

1. 解释 Adam 为什么同时维护 m_t 与 v_t ；
2. 正确解释 first moment / second moment，而不是把 v_t 简单说成“方差”；
3. 逐项解释 $\beta_1, \beta_2, \epsilon, \eta$ ；
4. 理解 zero initialization 为什么带来 early-step bias，以及 bias correction 为什么需要；
5. 手算 3-5 Step Adam；
6. 解释 Adam 为什么对不同 Gradient 尺度具有坐标级自适应；
7. 区分 nominal learning rate 与真正的 per-parameter effective update；
8. 看懂 PyTorch `torch.optim.Adam` 的状态变量；
9. 理解 `optimizer.state_dict()` 对断点续训的重要性；
10. 为下一讲 AdamW / Weight Decay 打好基础。

1 从 Momentum 到 Adam：还缺什么？

Momentum 的核心是：

$$v_t = \mu v_{t-1} + g_t$$

它能“记住过去的方向”。但想象两个参数坐标：

$$g = [100, 1]$$

如果普通 SGD 用同一个学习率 $\eta = 0.1$:

$$\Delta\theta = [-10, -0.1]$$

第一个坐标更新幅度是第二个的 100 倍。

这不一定错，但它暴露了一个问题：**不同坐标的 Gradient 数值尺度可能完全不同**。这种差异可能来自输入尺度、参数化方式、网络深度、局部曲率等，而我们却只用一个全局 LR。

原理

Adam 的核心思想可以概括为两句话：

1. 用 m_t 记住 Gradient 的“带符号方向历史”；
2. 用 v_t 记住 Gradient 平方的“尺度历史”，并据此做坐标级归一化。

2 Adam 的完整公式先看全貌

标准 Adam：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

因为从 0 初始化，要做 bias correction：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$

最终参数更新：

$$\theta_t = \theta_{t-1} - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

先不要被公式吓住

这 5 行公式只做三件事：**方向平滑、尺度估计、按尺度归一化后更新**。后面逐项拆开。

3 m_t ：First Moment 到底是什么？

m_t 是 Gradient 的指数移动平均（EMA）：

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

典型值：

$$\beta_1 = 0.9$$

它保留 Gradient 的符号，因此主要刻画“近期平均方向”。

与上一讲 Momentum 的联系非常直接。Momentum buffer 常写为：

$$b_t = \mu b_{t-1} + g_t$$

Adam 的 m_t 多了 $(1 - \beta_1)$ 缩放，使它更像标准 EMA，但本质都是：不要只相信当前 Step，而是融合历史。

连续同方向 Gradient

若 $g_t = 2$ 恒定， $\beta_1 = 0.9$ ，且 $m_0 = 0$ ：

$$m_1 = 0.2, \quad m_2 = 0.38, \quad m_3 = 0.542$$

它逐渐逼近 2，但前几步明显偏小。这就是稍后 bias correction 要解决的问题。

4 v_t ：Second Moment 不是简单的“方差”

Adam 同时维护：

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

典型值：

$$\beta_2 = 0.999$$

这里保存的是 Gradient 平方的指数移动平均。

为什么平方？

- 去掉正负号，只关注尺度；
- 大 Gradient 平方后贡献更明显；
- 最后通过 $\sqrt{\hat{v}_t}$ 把量纲拉回 Gradient 的尺度。

常见误区： v_t 就是方差

严格来说，Adam 的 v_t 通常是 **uncentered second moment estimate**，即 $E[g^2]$ 的移动估计，不是统计学中 $E[(g - E[g])^2]$ 的中心化方差。日常口语说“梯度方差”容易理解错，科研写作中最好准确表述。

5 为什么要除以 $\sqrt{\hat{v}_t}$ ？

最终更新中的关键部分：

$$\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

可以把它理解为：

“方向趋势”除以“最近这个坐标典型 Gradient 的尺度”。

若某坐标长期 Gradient 很大，则 $\hat{v}_t$ 也大，分母大，该坐标的有效更新被压缩；若某坐标 Gradient 较小，分母也小，更新不会单纯按绝对 Gradient 尺度被压得很小。

一个极端但直观的第一步

设：

$$g = [100, 1], \quad \eta = 0.1$$

SGD：

$$|\Delta\theta| = [10, 0.1]$$

Adam 第 1 Step (bias correction 后, 忽略 ϵ):

$$\hat{m} = g, \quad \hat{v} = g^2$$

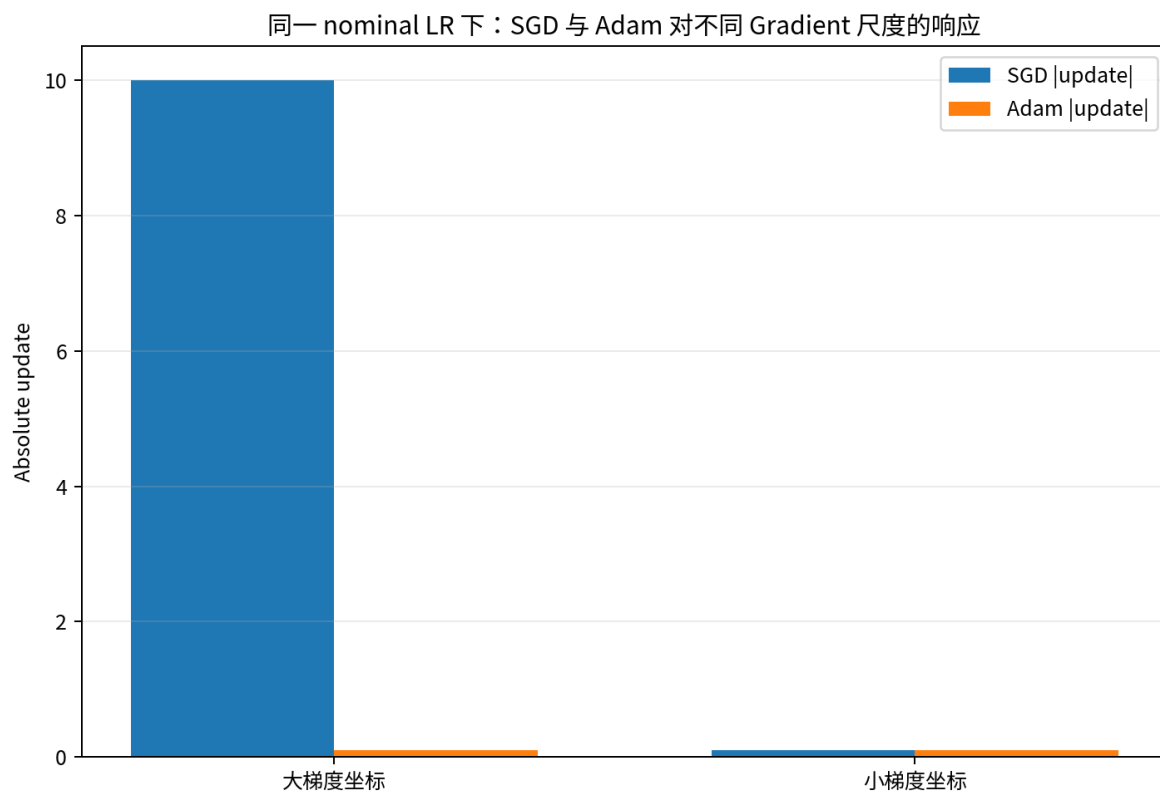
所以：

$$\frac{\hat{m}}{\sqrt{\hat{v}}} \approx [1, 1]$$

最终：

$$|\Delta\theta| \approx [0.1, 0.1]$$

这展示了 Adam 的“坐标级尺度自适应”。但不要误解为每一步永远都是 $\pm\eta$ ，历史 m, v 会让后续行为更复杂。



6 Bias Correction：为什么初始化为 0 会带来偏差？

Adam 通常：

$$m_0 = 0, \quad v_0 = 0$$

这会导致早期 EMA 被 0 “拖低”。例如真实 Gradient 恒为 2：

$$m_1 = (1 - 0.9) \times 2 = 0.2$$

明显不是 2。

因为：

$$E[m_t] = (1 - \beta_1^t)E[g]$$

所以除以：

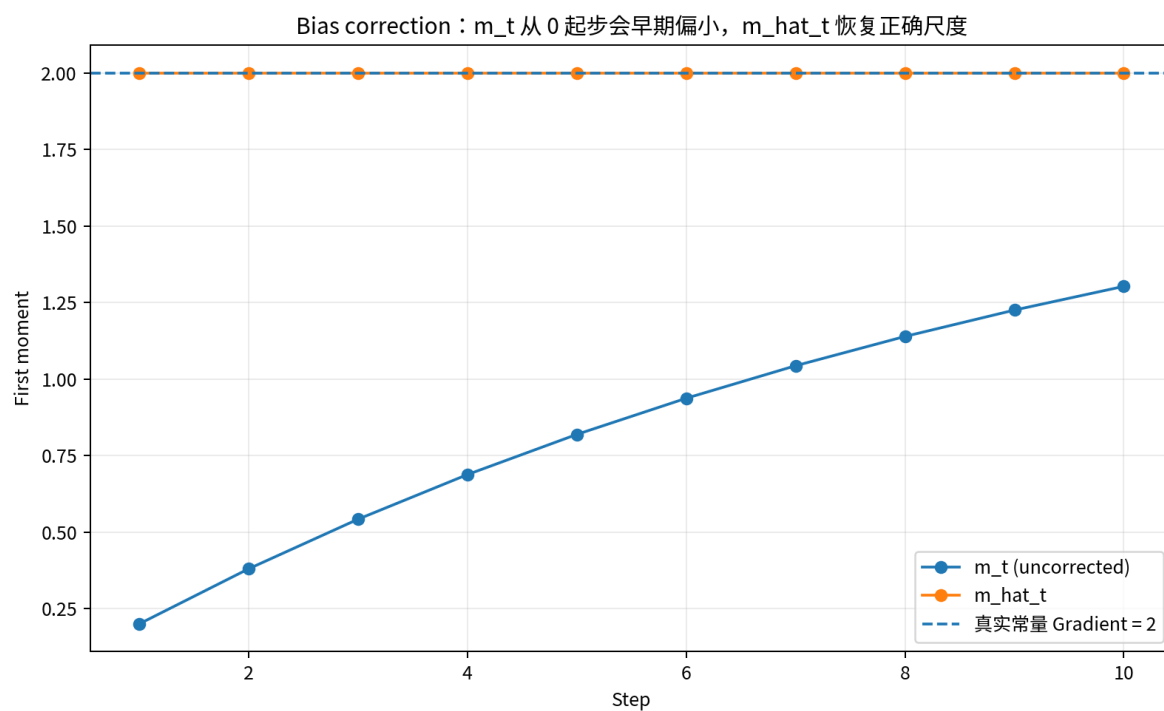
$$1 - \beta_1^t$$

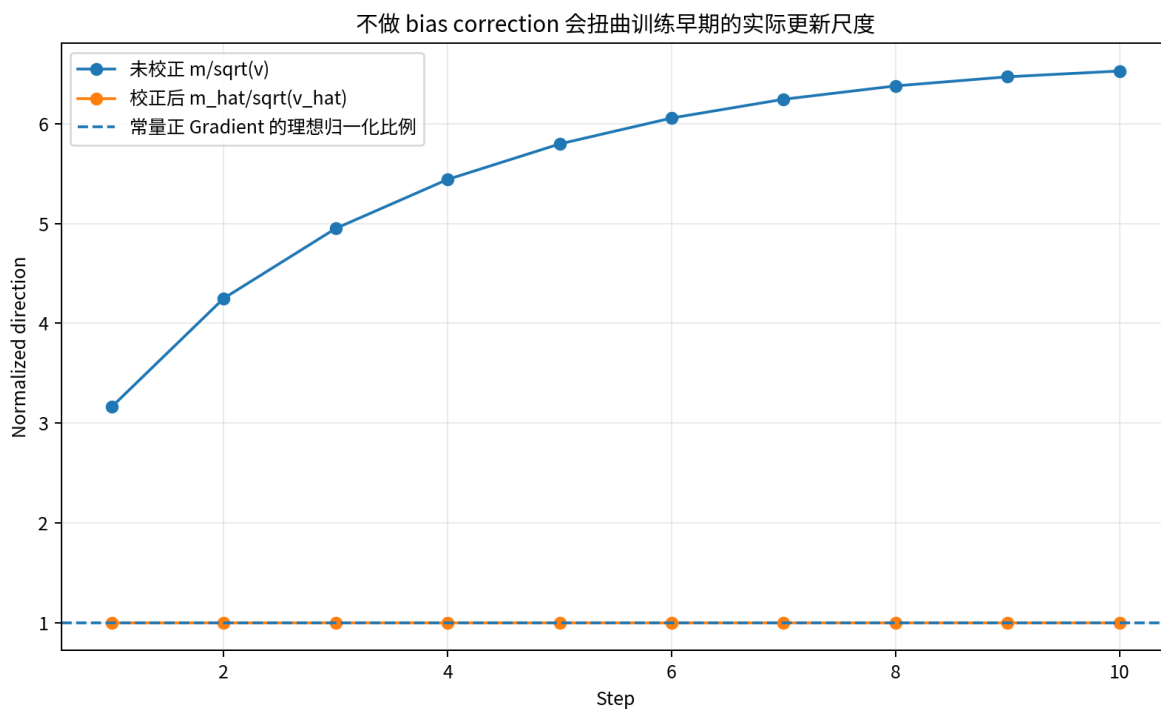
得到：

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}$$

同理：

$$\hat{v}_t = \frac{v_t}{1 - \beta_2^t}$$





原理

Bias correction 不是“为了让公式更漂亮”。它补偿 EMA 从 0 初始化造成的训练早期系统性偏低。如果不校正, m_t 和 v_t 的偏差还会以不同速度消失, 最终会扭曲实际更新尺度。

7 一个 5 Step 的完整手算

设:

$$\theta_0 = 5, \quad \eta = 0.1, \quad \beta_1 = 0.9, \quad \beta_2 = 0.999$$

Gradient 序列:

$$[2, 2, 1, -1, 0.5]$$

第 1 Step:

$$m_1 = 0.9 \times 0 + 0.1 \times 2 = 0.2$$

$$v_1 = 0.999 \times 0 + 0.001 \times 4 = 0.004$$

$$\hat{m}_1 = 0.2 / (1 - 0.9) = 2$$

$$\hat{v}_1 = 0.004 / (1 - 0.999) = 4$$

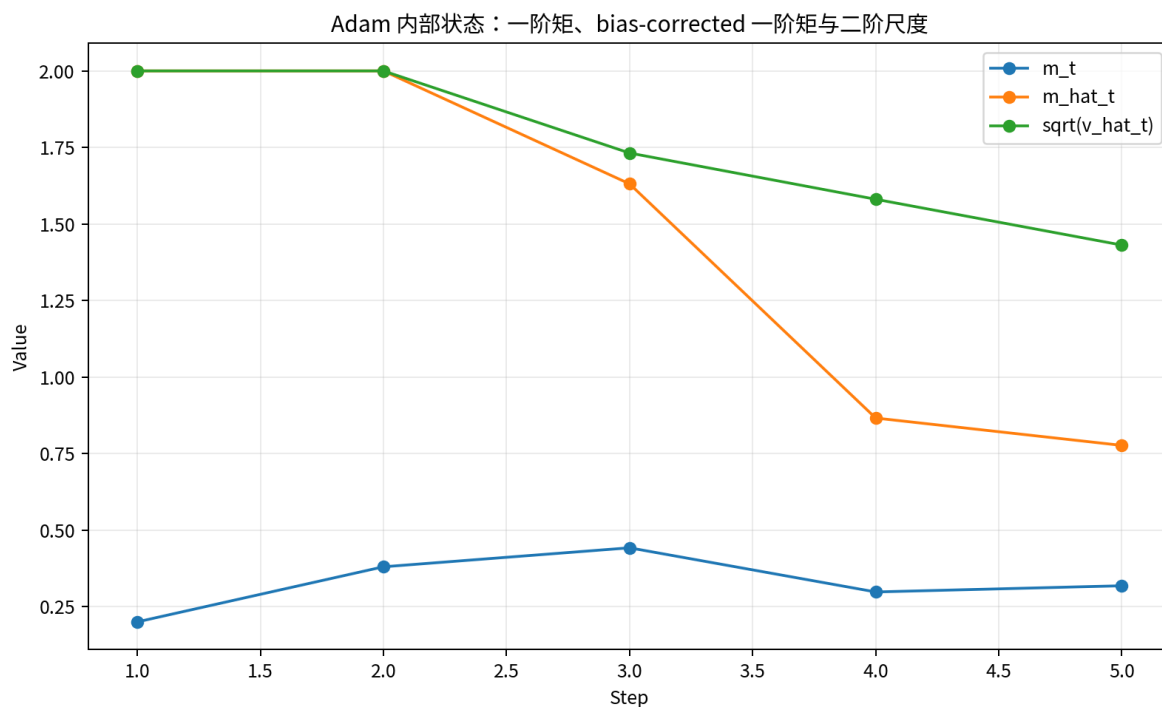
$$\Delta\theta_1 = 0.1 \times \frac{2}{\sqrt{4} + 10^{-8}} \approx 0.1$$

$$\theta_1 = 4.9$$

实验继续计算得到:

Step	Gradient	$\hat{m}_t$	$\hat{v}_t$	Update
1	2.0	2.0000	4.0000	0.1000
2	2.0	2.0000	4.0000	0.1000
3	1.0	1.6310	2.9990	0.0942
4	-1.0	0.8659	2.4985	0.0548
5	0.5	0.7766	2.0479	0.0543

注意第 4 Step 当前 Gradient 已经变成 -1 ，但 $\hat{m}_4$ 仍是正数。这正是历史方向记忆的表现：Adam 不会因为一次 Gradient 翻转就立刻完全转向。



8 β_1 与 β_2 分别在“记忆”什么？

典型默认值：

$$\beta_1 = 0.9, \quad \beta_2 = 0.999$$

β_1 控制 Gradient 方向历史的记忆； β_2 控制 Gradient 平方尺度历史的记忆。

粗略记忆长度常用：

$$\frac{1}{1 - \beta}$$

于是：

$$\beta_1 = 0.9 \Rightarrow \text{约 } 10 \text{ Step 量级}$$

$$\beta_2 = 0.999 \Rightarrow \text{约 } 1000 \text{ Step 量级}$$

这只是直觉，不是硬截断。

为什么 β_2 往往比 β_1 大?

Gradient 的方向可以相对快地响应最新信息，而二阶尺度估计若变化太剧烈，会让有效学习率本身也剧烈波动。因此常让 second moment 更平滑、更长记忆。但实际最优设置依任务而变，不能把默认值当物理常数。

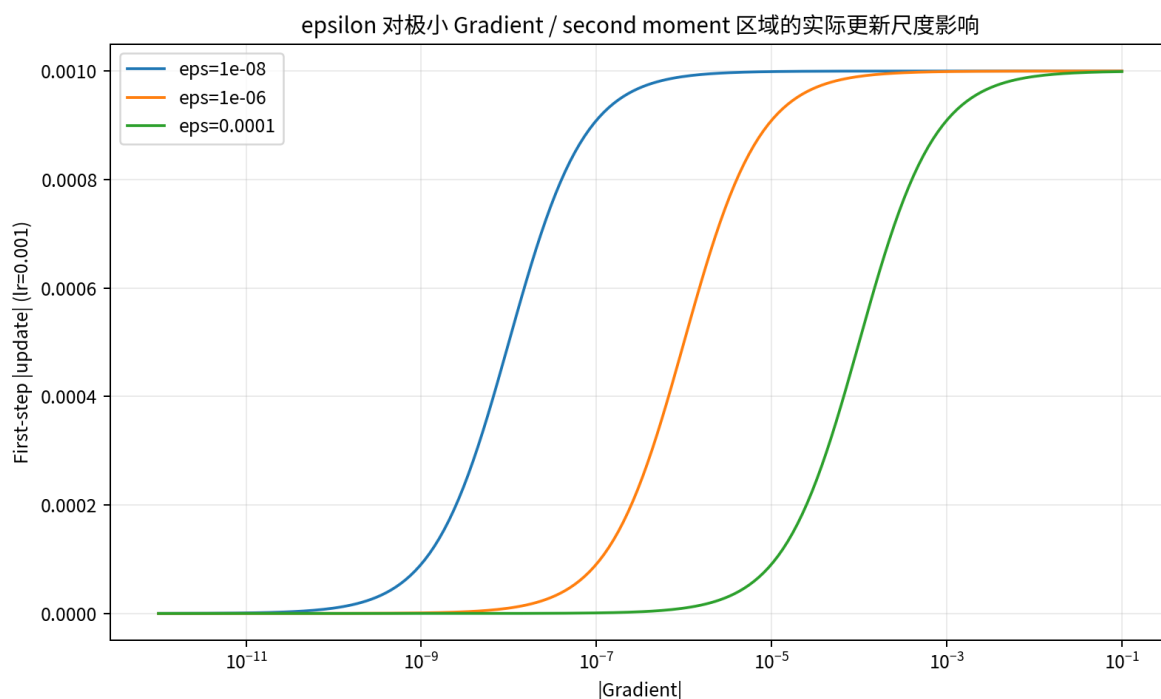
9 ϵ : 不只是“防止除以 0”

更新:

$$\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

ϵ 确实防止分母为 0，但当 $\sqrt{\hat{v}_t}$ 很小时，它还会实质影响有效步长。

实验中固定 $lr = 0.001$ ，当 Gradient 极小时，不同 ϵ 会产生明显不同的第一步 update。



常见误区

不要把 ϵ 当成完全无关紧要的数值小常数。多数正常区域它影响很小，但在极小 second moment、低精度数值环境或特定参数组中，可能影响真实更新。

10 Adam 的“有效学习率”不是一个单一数字

SGD:

$$\Delta\theta_i = -\eta g_i$$

Adam:

$$\Delta\theta_i = -\eta \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \epsilon}$$

因此不同参数坐标 i 的缩放因子不同。

所以论文写：

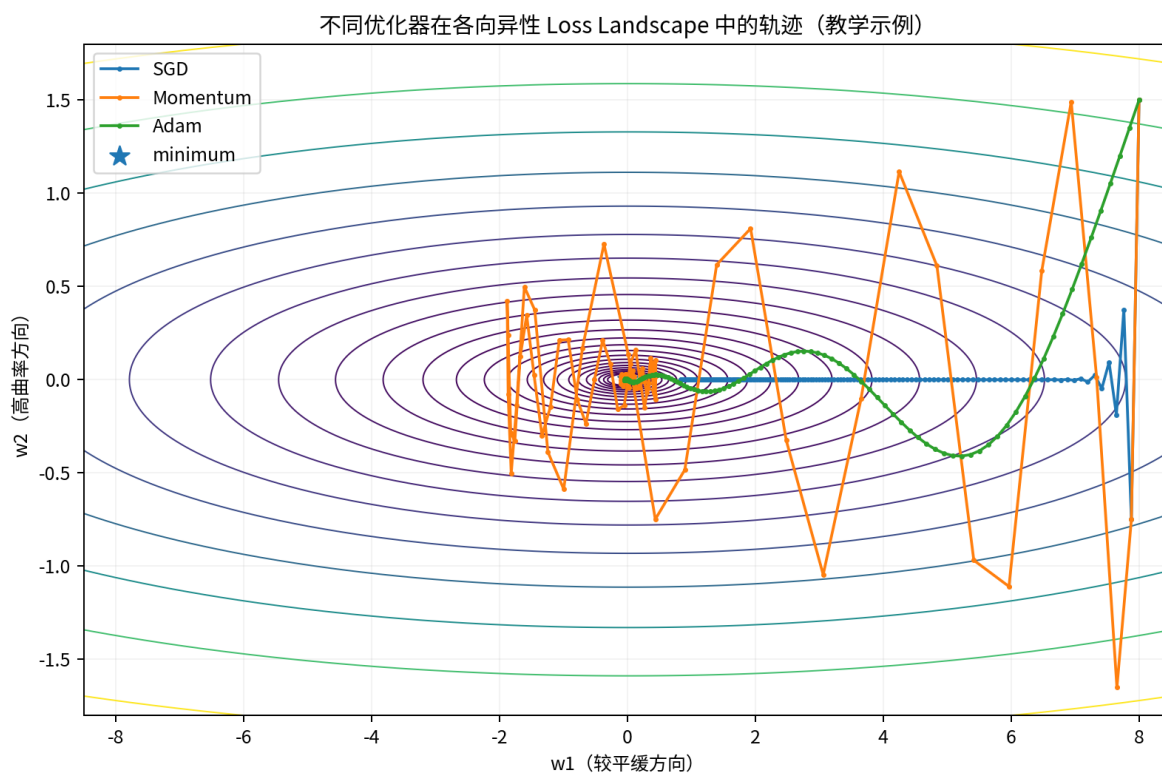
Adam, learning rate 3×10^{-4}

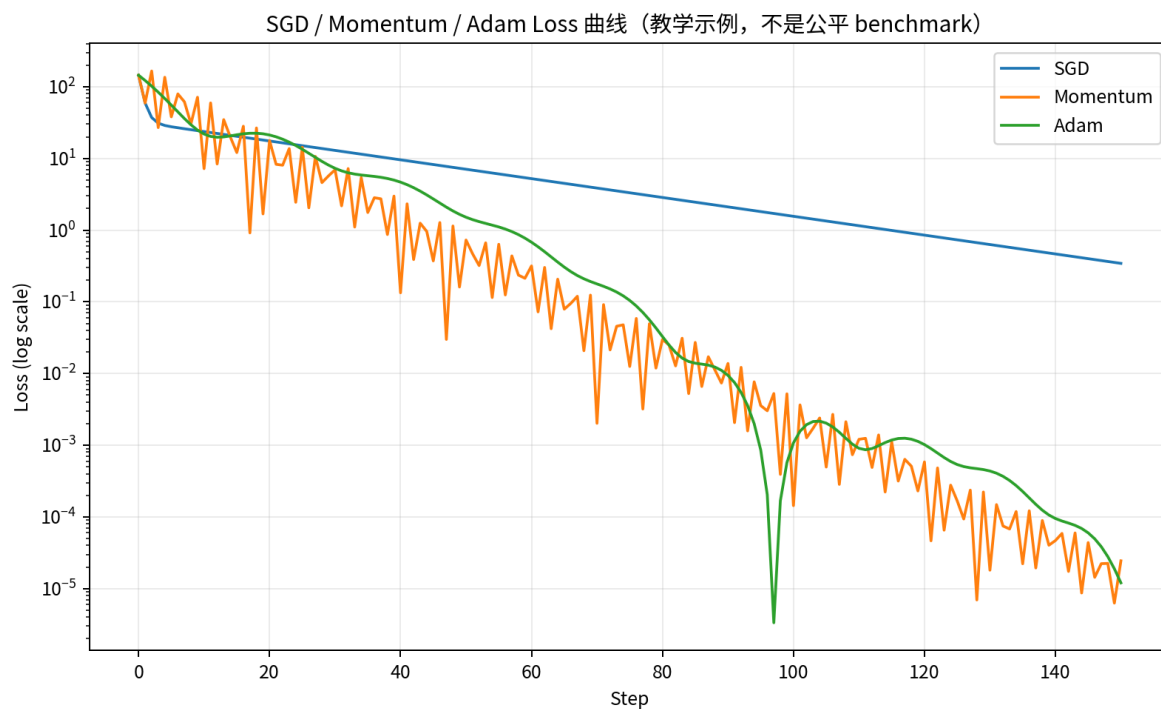
这个 3×10^{-4} 是 **nominal LR**，不是说每个参数每一步都移动这么多。

11 SGD、Momentum、Adam：三者到底差在哪？

	SGD	Momentum	Adam
当前 Gradient	使用	使用	使用
方向历史	无	有	有 (m_t)
尺度历史	无	无	有 (v_t)
坐标级自适应	无	无	有
Optimizer state	很少	buffer	$m, v, step$

本讲教学示例在各向异性二次 Loss 上画出了三类轨迹：





不能从这张图得出 “Adam 普遍最好”

本实验为了展示行为差异，为不同优化器选择了适合教学展示的 LR。这不是公平 benchmark。真实比较必须控制训练预算、调参范围、随机种子、Batch、Scheduler、Weight Decay 等。优化器优劣高度依赖任务和配置。

12 PyTorch 中的 Adam 到底保存了什么？

基本写法：

```
optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-3,
    betas=(0.9, 0.999),
    eps=1e-8,
)
```

第一次 `optimizer.step()` 后，每个 Parameter 会逐渐拥有 optimizer state。典型包括：

- `step`: 已经更新多少次；
- `exp_avg`: 对应 m_t ；
- `exp_avg_sq`: 对应 v_t 。

本讲用 float64 手写 Adam 与 PyTorch 连续对齐 8 Step，最大差异仅约 10^{-16} ，说明公式与框架实现对得上。

为什么 Adam 比 SGD 更“占状态”？

对于每一个可训练参数，Adam 除了 Parameter 本身和 Gradient，通常还需要维护同 Shape 的一阶矩与二阶矩状态。因此 optimizer state 的内存成本不可忽视。大模型训练中，Optimizer State 往往是显存/内存规划的重要部分。

13 为什么断点续训要保存 Optimizer State？

如果只保存：

```
torch.save(model.state_dict(), "model.pt")
```

你保存了 Parameter，但没有保存 Adam 已经积累的：

$$m_t, \quad v_t, \quad step$$

恢复后重新创建 Adam，这些状态又从 0 开始，训练轨迹会改变。

完整 checkpoint 通常还会包含：

- model state;
- optimizer state;
- scheduler state;
- global step / epoch;
- 随机数状态（高可复现要求时）。

14 Adam 与 Gradient Accumulation 的关系

第 34–35 讲已经学过 Gradient Accumulation。关键点：Adam 的 m_t, v_t 通常在 **optimizer step** 时更新，不是在每个 micro-batch backward 后更新。

例如 4 个 micro-batch 累积一次：

```
zero_grad
micro1 backward
micro2 backward
micro3 backward
micro4 backward
optimizer.step() # Adam state 更新 1 次
```

所以论文中的 t 、bias correction、scheduler step 等通常应理解为 **optimizer update index**，不是 forward 次数。

15 Adam 与 Batch Size / Gradient Noise

Adam 自适应缩放并不会让 Batch Size 变得无关紧要。

- 小 Batch: Gradient Noise 更强, m_t, v_t 都在追踪更噪声的信号;
- 大 Batch: Gradient 更稳定, 但 optimizer step 数和训练预算可能改变;
- 改 Batch Size 时, LR、 β 、warmup、训练 steps 可能需要联合考虑。

16 与 Transformer / LLM 的关系

你以前在 LLM 课程里见过 AdamW。现在可以把它拆成两层:

1. Adam: 本讲的 adaptive moment estimation;
2. W: 下一讲重点, decoupled weight decay。

大型 Transformer 训练中, 很多参数块的 Gradient 尺度和统计特征不同, Adam 类方法的坐标级自适应是其广泛使用的重要原因之一。但工程上还会联合:

- warmup;
- cosine / linear decay;
- gradient clipping;
- mixed precision;
- weight decay;
- parameter group;
- distributed optimizer state sharding。

17 与 Agent Systems 科研的关系

很多 Agent Systems 研究不会从零预训练基础模型, 但你仍可能:

- 微调 policy / reranker / reward model;
- 训练小型 routing / selection module;
- LoRA / QLoRA 微调;
- 复现实验中的 optimizer 配置;
- 判断训练失败是数据、Loss、LR 还是 optimizer state 问题。

科研阅读时不要只抄 “Adam lr=...”

至少一起记录：Adam 还是 AdamW、 β_1 、 β_2 、 ϵ 、LR schedule、warmup、weight decay、global batch / tokens per update、gradient clipping、total optimizer steps。否则 “同一个 optimizer 名称” 并不足以复现实验。

18 训练异常时怎样诊断 Adam?

1. **Loss 一开始就爆炸**：先看 LR、mixed precision、Gradient clipping、数据/目标是否异常；
2. **训练非常慢**：看 LR 是否过小、 v_t 是否让某些方向有效步长很小、scheduler 是否提前衰减；
3. **恢复 checkpoint 后曲线突然变**：检查 optimizer state / scheduler state 是否一起恢复；
4. **改 Batch 后结果变化大**：不能只怪 Adam，要检查 Gradient Noise、optimizer steps、LR schedule、effective batch；
5. **参数几乎不动**：看 Gradient 是否为 0/None、LR、 $\hat{m}/\sqrt{\hat{v}}$ 的尺度、参数是否真的在 optimizer parameter group 中。

19 常见误区总结**误区 1：Adam 就是 “Momentum + 自动调 LR”**

这个说法可作为第一直觉，但不够精确。Adam 的自适应是逐坐标、由一阶和二阶矩共同决定的，且有 bias correction 与 ϵ 。

误区 2： v_t 是 Gradient 方差

更准确是 Gradient 平方的指数移动平均 / uncentered second moment estimate。

误区 3：Adam 不需要调 Learning Rate

错误。Adam 仍然有全局 nominal LR，而且 LR 往往是最敏感的超参数之一。

误区 4：Adam 永远比 SGD 好

错误。任务、模型、泛化目标、调参预算都可能改变结论。不要从单一教学图推广为普遍规律。

误区 5：zero_grad 会把 Adam 的历史也清掉

错误。zero_grad() 清 Parameter Gradient； m_t, v_t 属于 optimizer state，会跨 Step 保留。

20 本讲总闭环

$$g_t \rightarrow \begin{cases} m_t : \text{方向历史} \\ v_t : \text{尺度历史} \end{cases} \rightarrow \text{bias correction} \rightarrow \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \rightarrow \eta \rightarrow \theta_t$$

最低闭环能力

学完本讲，你不需要背每个推导细节，但必须能看到 `torch.optim.Adam` 时解释：它维护哪些状态、为什么有 β_1/β_2 、为什么需要 bias correction、为什么不同参数的实际步长不同、为什么断点续训需要 optimizer state。

21 把 Adam 看成“每个参数坐标都有自己的缩放器”

如果参数是一个向量：

$$\theta = [\theta_1, \theta_2, \dots, \theta_d]$$

Adam 实际上为每个坐标维护自己的：

$$m_{t,i}, \quad v_{t,i}$$

所以更新是逐坐标的：

$$\Delta\theta_i = -\eta \frac{\hat{m}_{t,i}}{\sqrt{\hat{v}_{t,i} + \epsilon}}$$

这和普通 SGD 很不一样。SGD 只有一个全局 η 去乘不同坐标的 Gradient；Adam 在全局 η 前面又乘了一个随坐标、随时间变化的归一化因子。

把“有效步长”拆开

假设某一坐标：

$$\hat{m} = 0.02, \quad \sqrt{\hat{v}} = 0.2, \quad \eta = 10^{-3}$$

忽略 ϵ ：

$$\Delta\theta = -10^{-3} \times \frac{0.02}{0.2} = -10^{-4}$$

另一个坐标即使 Gradient 绝对值更大，如果它的历史二阶尺度也同比更大，最后实际更新未必更大。

这就是为什么不能只盯着 Gradient norm 来猜 Adam 的 parameter update norm。真正 update 还经过了 first moment、second moment、bias correction、 ϵ 和 LR。

22 为什么是“平方再开方”，而不是直接取绝对值？

从直觉上，我们需要一个始终非负的“尺度估计”。 g^2 满足：

- 正负 Gradient 都变成非负；
- 大 Gradient 的影响更明显；
- 对 g^2 做 EMA 后，再开平方，可以恢复到和 Gradient 相同的量纲。

如果不平方，直接对 Gradient 做平均，正负会抵消，那已经是 m_t 在做的事情；如果只维护 $|g|$ ，也可以构造其他优化器，但标准 Adam 选择二阶矩形式。

量纲直觉

g 的单位假设是 “Loss / Parameter”，则 g^2 的单位是平方。 $\sqrt{v}$ 再把单位还原，因此 $m/\sqrt{v}$ 是一种无量纲或近似尺度归一化的方向量。这个直觉有助于理解为什么开平方出现在分母。

23 当 Gradient 突然变号时，Adam 会怎样？

实验 1 的第 4 Step：当前 Gradient 从正变成负：

$$g_4 = -1$$

但由于前面长期为正：

$$\hat{m}_4 \approx 0.866 > 0$$

所以这一步实际 update 仍沿历史正方向。

这说明 Adam 不是：

“看到当前 Gradient 是负就马上反向。”

而是：

“当前 Gradient 是新证据，但要和历史方向共同决定下一步。”

如果后面连续很多步都为负， m_t 才会逐步翻转。这与 Momentum 的惯性直觉一致。

24 当 Gradient 尺度突然变大时， v_t 会怎样？

假设某个坐标长期 Gradient 约为 0.1，突然某一步变成 10。由于平方：

$$0.1^2 = 0.01, \quad 10^2 = 100$$

这个异常大 Gradient 会显著抬高 v_t ，使之后一段时间分母变大，从而降低该坐标的有效更新尺度。

但 $\beta_2 = 0.999$ 记忆很长，所以这种尺度冲击也可能持续较久。这说明：

- β_2 太小：尺度估计变化太快，update 可能更抖；
- β_2 太大：尺度估计反应慢，异常历史影响保留更久。

不存在脱离任务的“绝对最佳 β_2 ”。

25 Bias Correction 再深入一步：为什么两个 correction 都要有？

很多初学者看到 m_t 、 v_t 都偏向 0，会想：

“反正分子分母都偏小，互相抵消不就行了吗？”

并不会，因为 β_1 与 β_2 通常不同。

第一步恒定正 Gradient 时：

$$m_1 = (1 - 0.9)g = 0.1g$$

$$v_1 = (1 - 0.999)g^2 = 0.001g^2$$

如果不校正：

$$\frac{m_1}{\sqrt{v_1}} = \frac{0.1g}{\sqrt{0.001}|g|} \approx 3.162 \operatorname{sign}(g)$$

并不是 1。

所以两个“都偏小”并不代表比例正确。Bias correction 的价值正是在训练初期把这两个不同时间尺度的 EMA 拉回合理尺度。

26 Adam 与“预条件”直觉

在优化理论里，可以把 Adam 粗略理解成一种对 Gradient 做**对角预条件（diagonal preconditioning）**的方法：不同坐标乘上不同的缩放因子。

普通 SGD：

$$\Delta\theta = -\eta Ig$$

Adam 的直觉形式：

$$\Delta\theta \approx -\eta D_t g_{\text{history}}$$

其中 D_t 是由 $1/(\sqrt{\hat{v}} + \epsilon)$ 构成的对角缩放。

这里不要求你深入数值优化理论，只要建立一个直觉：

Adam 不仅改方向的时间平滑，还在改变不同坐标轴的尺度。

27 Adam 的“尺度不敏感”也有边界

第 1 Step 中，把 Gradient 从 $[1, 1]$ 整体放大成 $[100, 100]$ ，忽略 ϵ 时归一化结果都接近 $[1, 1]$ 。这容易让人误以为 Adam 完全不在乎 Gradient scale。

但真实训练中仍受以下因素影响：

- m_t, v_t 有历史；
- ϵ 在极小尺度区域会参与；
- Gradient clipping 会先改变 Gradient；
- mixed precision / loss scaling 会影响数值路径；
- weight decay（下一讲）会额外改变参数；
- 参数组可能使用不同 LR 或不同 betas。

所以“Adam 自动解决所有尺度问题”是过度简化。

28 把一次 PyTorch Adam Step 拆成时间顺序

标准训练循环：

```
optimizer.zero_grad()
pred = model(x)
loss = loss_fn(pred, y)
loss.backward()
optimizer.step()
```

底层概念顺序：

1. `zero_grad()`：清 Parameter 的旧 Gradient；
2. Forward / Loss：建立本次计算图并得到标量目标；
3. `backward()`：得到当前 g_t ；
4. `step()` 内部更新 m_t ；
5. 同一 `step()` 内部更新 v_t ；
6. 做 bias correction；
7. 根据 $\hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$ 修改 Parameter；
8. optimizer state 保留下来进入下一 Step。

所以 `backward()` 并不知道 Adam；Autograd 只负责 Gradient。Adam 的逻辑发生在 Optimizer 的 `step()` 中。

29 Parameter Group：同一个 Adam 里也可以有不同超参数

PyTorch Optimizer 可以把参数分组，例如某些参数用不同 LR 或 weight decay：

```
optimizer = torch.optim.Adam([
    {"params": group1, "lr": 1e-3},
    {"params": group2, "lr": 1e-4},
])
```

因此看到“optimizer=Adam”仍然不够，需要检查 parameter groups。LLM 微调中常见：不同模块、不同参数类型可能拥有不同训练策略。

30 Optimizer State 的内存直觉

假设模型有 P 个 float32 参数。非常粗略地看：

- Parameter：约 $4P$ bytes；
- Gradient：约 $4P$ bytes；

- Adam first moment: 约 $4P$ bytes;
- Adam second moment: 约 $4P$ bytes。

还没计算 mixed precision master weights、activation、temporary buffers 等，就已经有多份与参数同规模的状态。

这就是大模型训练中为什么 optimizer state sharding、ZeRO 等系统技术会重要。当前课程不深入分布式训练，但你以后看到“optimizer state memory”时应知道它从哪里来。

31 Adam 会不会让模型 Generalization 更好?

不能从“训练更容易”直接推出“测试更好”。Optimizer 同时影响:

- 优化速度;
- 参数最终落在哪个区域;
- 隐式正则化特性;
- 与 LR schedule / weight decay 的耦合。

某些任务 SGD 可能有更好的最终泛化，某些任务 Adam/AdamW 更实用。必须以具体实验和训练预算为依据。

论文比较的最低规范

如果比较 Optimizer，不应只固定“同一个 LR”就宣布公平。至少要考虑：各优化器合理的 LR 搜索范围、Scheduler、Weight Decay、训练 Steps、Batch、Warmup、随机种子和计算预算。否则很容易比较的是“超参数匹配程度”，不是优化器机制本身。

32 怎样从训练日志观察 Adam 是否可能异常?

如果代码允许记录，可以关注:

- train / validation loss;
- gradient norm;
- parameter update norm;
- update-to-weight ratio;
- learning rate;
- 是否出现 NaN / Inf;
- 某些关键层的 $\|m_t\|$ 、 $\|v_t\|$ 或统计量。

这里只是诊断线索，不要机械使用单一阈值。不同模型层、不同参数组的尺度可以非常不同。

与下一讲的连接

Adam 解决的是：如何利用 Gradient 的一阶和二阶历史来更新参数。但我们还没有真正讲清 **Weight Decay** 应该怎样和 Adam 结合。下一讲进入：

第 39 讲：AdamW 与 Weight Decay——为什么“大模型常用 AdamW”而不是简单 Adam + L2。